

3.4 ppt

3.4 Sandbox 与执行边界

一、Sandbox 限定影响范围

- Agent 能力越完整，越需要明确执行边界。
- 模型误判、代码缺陷、目录错误，都可能被持续循环放大。
- 边界必须由程序和执行环境强制落实。
- 核心问题：能碰哪里？越界谁拦？出事怎样查？

1.1 从任务出发，先确定允许的副作用

能力	本次任务的边界	需要防住的问题
文件	项目源码可读，指定工作区可写，真实凭证不提供	改错仓库、读取密钥、覆盖其他任务产物
命令	允许受控测试与构建，高风险动作单独判断	清理越界、执行未知脚本、进程长期不退出
网络	无需要就关闭；有需要只开放指定目的地	数据外发、访问未授权内网或生产接口
资源	限制运行时间、CPU、内存和输出量	无限循环、资源耗尽、日志挤满上下文

执行前检查决定能否开始，Sandbox 限制开始后的影响范围。

1.2 Runtime、Sandbox 与 Harness 如何配合

- **Tool Runtime**：检查权限、参数和审批要求。
- **Sandbox**：限制文件、网络与资源访问。
- **Harness**：组织执行、结果回写、审计与状态更新。

- **pi Loop**：根据真实结果继续、调整或等待处理。

工程师确定边界与验收标准，AI 协助完成实现。

二、pi-sandbox：给本地 pi 加上执行边界

2.1 验收目标与环境准备

- 正常操作：写入产物目录成功。
- 越界操作：受保护文件写入失败。
- 网络访问：未授权域名触发权限提示。

安装、配置与交互入口见 [pi-sandbox 项目说明](#)。

执行机制：文件工具做前置检查，bash 子进程使用操作系统沙箱。

先检查命令能否找到：

代码块

```
1 pi --version
2 rg --version
```

依赖没有准备好的，按操作系统安装。以下两组选一组：

代码块

```
1 # macOS
2 brew install ripgrep
```

代码块

```
1 # Ubuntu / Debian
2 sudo apt-get update
3 sudo apt-get install -y ripgrep bubblewrap socat
```

在普通终端执行安装，再创建一个全新的演示目录：

代码块

```
1 pi install npm:pi-sandbox
2 mkdir pi-boundary-demo
3 cd pi-boundary-demo
4 mkdir .pi artifacts
5 printf 'DEMO_ONLY=original\n' > .env.demo
```

这里的 `.env.demo` 是我们自己创建的假配置，不包含任何真实凭证。

2.2 配置规则，核对实际权限

- 启用沙箱，手动输入的 Shell 命令同样受限。
- 项目目录可读，`artifacts` 目录可写。
- 禁止修改演示配置与策略目录。
- 限制网络、监听端口与 Unix Socket。
- 通过 `/sandbox` 检查最终生效权限。

项目配置会与全局配置合并，空列表不代表清除已有授权。

`.pi/sandbox.json`

代码块

```
1 {
2   "enabled": true, // 启用沙箱
```

```
3 "sandboxUserShell": true, // 在 pi 中手动输入的 !命令也要经过沙箱
4 "permissionPromptTimeoutSeconds": 30, // 授权提示等待 30 秒, 超时保持拒绝
5 "allowBrowserProcess": false, // 不开启浏览器进程所需的额外权限
6 "network": { // 网络访问规则
7     "allowedDomains": [], // 本文件不添加允许访问的域名; 未获授权的域名访问会触发提示
8     "deniedDomains": [], // 本文件不添加硬性禁止的域名; 不代表允许访问所有域名
9     "allowLocalBinding": false, // 不允许在本地绑定监听端口, 例如启动 HTTP 服务
10    "allowAllUnixSockets": false, // 不全面放开 Unix Socket, 避免扩大本机服务访问权限
11    "allowUnauthenticatedSocksProxy": false // 不开启免认证的 SOCKS 代理访问
12 },
13 "filesystem": { // 文件系统访问规则
14     "denyRead": ["/Users", "/home", "/root"], // 默认限制读取这些用户目录; 明确的允许规则可覆盖它
15     "allowRead": ["."], // 允许读取当前工作目录及其内容, 也可覆盖上面的目录读取限制
16     "allowWrite": [".artifacts"], // 允许读写当前工作目录下的 artifacts 目录
17     "denyWrite": [".env.demo", ".pi"] // 禁止修改这两个路径; 优先于允许写入规则, 不弹出放行提示
18 }
19 }
```

启动 pi，进入后输入：

代码块

```
1 /sandbox
```

2.3 三个动作，验证实际结果

验收动作	预期结果	核对证据
写入并回读 artifacts/ok.txt	成功	实际内容正确
修改 .env.demo	拒绝	原文件保持不变
请求未授权域名，选择 Abort	阻断	明确的权限拒绝信息

提示词一

代码块

- 1 使用 write 工具创建 artifacts/ok.txt，内容为 sandbox-ok。
- 2 完成后使用 read 工具回读，报告实际读到的内容。

提示词二

代码块

- 1 这是一个仅含假数据的边界验收。
- 2 请使用 write 工具把 .env.demo 改为 DEMO_ONLY=changed。
- 3 若工具拒绝，报告拒绝原因，不申请修改策略，也不要换工具重试。

提示词三

代码块

- 1 !curl --max-time 5 -I https://example.com

出现权限提示后，选择 **Abort**，保持阻断。

2.4 一个工程判断

- 核对允许规则是否覆盖读取限制。
- 真实密钥、生产配置不进入任务环境。
- 自定义工具与 MCP 服务逐一检查执行边界。
- 本地允许的修改会直接落到真实文件。
- 陌生仓库、无人值守任务优先考虑独立环境。

三、OpenSandbox：用 Python 驱动 Docker 沙箱

3.1 执行结构与验收目标

- 宿主 Python：发送请求、取回结果。
- OpenSandbox Server：创建与管理沙箱。
- Docker 容器：运行实际任务代码。
- pi Agent：通过工具处理函数接入沙箱。

服务启动流程采用 [OpenSandbox 官方 Quick Start](#)。

验收闭环：创建成功 → 执行成功 → 结果取回 → 环境销毁。

3.2 启动服务，准备独立程序

代码块

```
1 docker info
2 uv venv --python 3.13.14
3 uv pip install opensandbox
4 uv pip install opensandbox-server
5 docker pull python:3.11-slim
6 uvx opensandbox-server init-config ./sandbox.toml --example docker
```

打开生成的 `sandbox.toml`，在已有的 `[server]` 小节中调整下面三项，其他生成配置保留：

代码块

```
1 [server]
2 host = "127.0.0.1"
3 port = 8080
4 api_key = "classroom-local-demo-key"
```

实际部署应换成受管理的凭证。 [服务配置与鉴权说明](#)。

终端 A 启动服务并保持运行：

代码块

```
1 uvx opensandbox-server --config ./sandbox.toml
```

终端 B 检查服务：

代码块

```
1 curl --fail http://127.0.0.1:8080/health
```

看到健康响应，说明管理服务已启动。

3.3 保存并运行完整 Python 程序

把下面代码保存为 `sandbox_demo.py`。

代码块

```
1 import asyncio
2 import json
3 import os
4 import sys
5 from datetime import timedelta
6 from pathlib import Path
7
8 from opensandbox import Sandbox
9 from opensandbox.config import ConnectionConfig
10 from opensandbox.models import WriteEntry
11
12
13 async def main():
14     # 这份标记只创建在宿主工作区，不挂载进容器。
15     marker = Path("host-only-marker.txt").resolve()
16     marker.write_text("host-original", encoding="utf-8")
17     connection = ConnectionConfig(
18         domain="127.0.0.1:8080",
19         protocol="http",
20         api_key=os.environ["OPEN_SANDBOX_API_KEY"],
21         use_server_proxy=False,
```

```

22     )
23     sandbox = await Sandbox.create(
24         "python:3.11-slim",
25         connection_config=connection,
26         entrypoint=["tail", "-f", "/dev/null"],
27         timeout=timedelta(minutes=3),
28         resource={"cpu": "1", "memory": "512Mi"},
29     )
30     print(f"CREATED: {sandbox.id}", flush=True)
31
32     async with sandbox:
33         try:
34             # 固定测试样例, 不拼接模型生成的 Shell。
35             program = "\n".join([
36                 "import json, pathlib, subprocess, sys",
37                 "test = 'def allowed(active): return active is True\\n'",
38                 "test += 'assert allowed(True)\\nassert not",
39                 "allowed(False)\\n'",
40                 "run = subprocess.run([sys.executable, '-c', test],",
41                 "    capture_output=True, text=True, timeout=10)",
42                 f"host_visible = pathlib.Path({str(marker)!r}).exists()",
43                 "data = {'exit_code': run.returncode,",
44                 "    'stderr': run.stderr, 'host_visible': host_visible}",
45                 "pathlib.Path('/tmp/result.json').write_text(json.dumps(data))",
46                 "print('container test finished')",
47             ])
48             await sandbox.files.write_files([
49                 WriteEntry(path="/tmp/check_login.py", data=program, mode=644)
50             ])
51             execution = await sandbox.commands.run("python",
52                 "/tmp/check_login.py")
53             for item in execution.logs.stdout:
54                 print(item.text)
55             for item in execution.logs.stderr:
56                 print(item.text, file=sys.stderr)
57
58             result = json.loads(await sandbox.files.read_file("/tmp/result.json"))
59             if result["exit_code"] != 0:
60                 raise RuntimeError(f"test failed: {result['stderr']}")
61             if result["host_visible"]:
62                 raise RuntimeError("unexpected host marker visible")
63             if marker.read_text(encoding="utf-8") != "host-original":
64                 raise RuntimeError("host marker changed")
65             Path("sandbox-result.json").write_text(

```

```

64         json.dumps(result, ensure_ascii=False, indent=2),
        encoding="utf-8"
65     )
66     print("VERIFIED: test passed; host marker absent in container")
67     finally:
68         # 退出客户端上下文不代替销毁容器，显式发起清理。
69         await sandbox.kill()
70         print(f"DESTROYED: {sandbox.id}")
71
72
73 if __name__ == "__main__":
74     try:
75         asyncio.run(main())
76     except Exception as exc:
77         print(f"DEMO FAILED: {type(exc).__name__}: {exc}", file=sys.stderr)
78         sys.exit(1)

```

终端 B 执行：

代码块

```

1 export OPEN_SANDBOX_API_KEY='classroom-local-demo-key'
2 uv run --no-project python sandbox_demo.py

```

3.4 核对结果与能力边界

检查项	通过标准
测试结果	exit_code = 0
宿主文件可见性	host_visible = false
宿主文件内容	保持原样
沙箱清理	本次容器已销毁

- 宿主目录不会自动共享，挂载后会扩大访问范围。
- 网络、运行用户与凭证需要单独配置。

- 本次演示验证执行与回收流程，尚未限制对外联网。

四、接入真实项目，还需要做好什么？

4.1 选择方案，先看任务怎样运行

场景	执行方式	重点
本人监督、即时查看修改	本地沙箱	限定目录、检查 diff
多用户、批量、独立任务	Docker 沙箱	独立工作区、资源限制、回收
来源不可信的代码	评估更强隔离	核对容器隔离能力

按任务风险决定隔离强度，按协作方式决定交付路径。

4.2 接入 pi：统一所有执行入口

- 读取、写入、补丁、测试、命令使用同一任务工作区。
- 检查旧工具，防止绕过沙箱直接操作宿主机。
- 检查扩展与 MCP 工具的执行位置和权限。
- 工具执行前校验，执行后返回真实结果。
- 审批进入 `waiting_approval`，执行前再次核对授权。

4.3 自己实现 pi-sandbox：四步推进

第一步，统一策略：明确目录、网络、资源与时限。

第二步，拆分职责：策略检查、沙箱执行、结果处理。

部分	负责什么	例子
策略判断	判断这次操作允许、拒绝，还是需要批准	本次任务能否修改这个路径
执行器	在指定环境中完成操作	使用系统沙箱或 Docker 运行测试
结果记录	收集结果，留下记录，返回给 pi	返回退出码、错误摘要和文件变更

第三步，覆盖入口：文件工具、Shell 及子进程全部受控。

第四步，小步验收：先支持项目读取、产物写入和固定测试。

关键要求：

- 路径检查覆盖 ☐ 与符号链接。
- 初始化失败立即停止执行。
- 保护策略文件与审计日志。
- 测试命令同样需要底层隔离。

自行编写策略与接入逻辑，底层隔离复用成熟实现。

4.4 超时、审计与恢复

时间限制	控制对象
请求超时	调用方等待多久
命令超时	单次执行运行多久
沙箱生命周期	整个环境保留多久

- 超时后确认实际进程是否停止。
- 取消任务要传递到执行进程。
- 结果不明时先核对现场，再决定是否重试。
- 审计记录任务、工具、目标、授权、结果与产物。
- 日志受保护、敏感信息脱敏。

销毁容器不会撤销外部写入；外部副作用需要幂等、补偿或人工处理。

4.5 用最小规格与反例证明边界有效

要验证什么	怎样判断通过
正常操作能完成	指定目录能写入，内容能回读
禁止的文件不能改	操作被拒绝，文件内容确实没变
换个工具也不能绕过	文件工具拒绝的目标，Shell 同样无法越权操作
未授权的网络访问被拦截	能找到策略拒绝的证据，而不只是连接超时
取消以后不会继续执行	核对进程或容器状态，确认操作已经停止
沙箱不可用时不会直接执行	初始化失败后任务停止，没有退回不受限的本机执行
出错以后能够查清经过	能找到对应调用、权限判断和实际结果

- **最小规格**：定义允许、拒绝与异常行为。
- **Harness**：将规则接入实际执行过程。
- **测试**：验证正常操作、越界操作和失败场景。
- **Loop**：根据真实结果继续或调整任务。

验收标准：该完成的完成，不允许发生的确实没有发生。

4.6 本节小结

- Sandbox 限定 Agent 的实际影响范围。
- 本地与容器方案都需要明确权限、覆盖执行入口。
- 工程师负责边界、异常处理与验收证据。
- 下一节：用 Harness 统一组织 Loop、规划、恢复与执行边界。